C-CLARKE INTERNATIONAL INSTITUTE OF DIGITAL SCIENCES

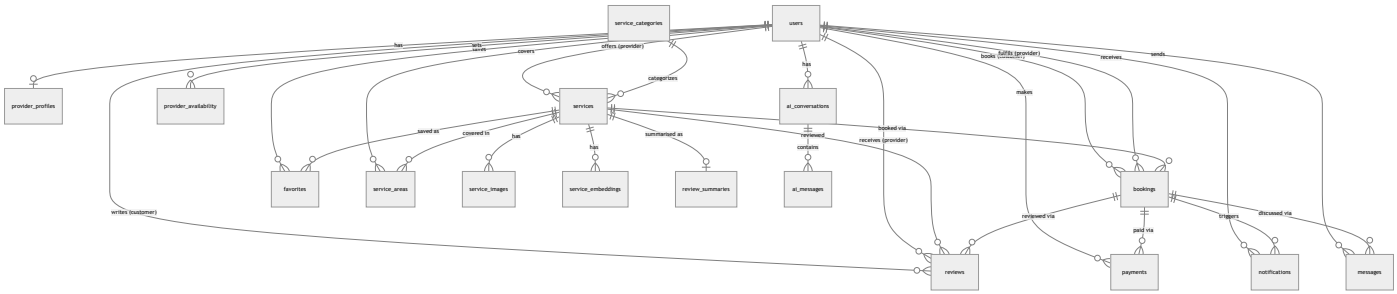
# Database Design

Entity-relationship diagram, table dictionary, and integrity rules for the live schema.

| | |
|---|---|
| **Program** | Diploma in Advanced AI & Software Engineering |
| **Batch** | 25/26 |
| **Project** | Smart Hire — Service Marketplace |
| **Team** | Nilan Iddawela, Janaka Eranda, Dasun Anjana, Damith Lasantha, Matheesha Milan |
| **Date** | 21 July 2026 |

# 1. Entity-Relationship Diagram

The live schema (PostgreSQL, hosted on Supabase) has 16 domain tables. Relationships are enforced with real foreign keys, not just application logic.




# 2. Table Dictionary

### users

Every account — customer, provider, or admin — in one table, distinguished by `role`.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | Auto-increment |
| `email` | varchar, unique | Login identifier |
| `password_hash` | varchar | bcrypt hash; never returned by the API |
| `full_name` | varchar | |
| `phone` | varchar, nullable | |
| `role` | enum | customer / provider / admin |
| `status` | enum | active / pending / suspended / deactivated |
| `email_verified` | boolean | |
| `created_at / updated_at` | timestamptz | |

### provider_profiles

One-to-one extension of `users` for provider-only fields.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `user_id` | integer, PK, FK → users.id | |
| `bio` | text, nullable | |
| `years_experience` | integer | |
| `verification_status` | enum | unverified / pending / verified / rejected |
| `avg_rating` | numeric | Recalculated whenever a review is created |
| `total_reviews` | integer | Recalculated whenever a review is created |

### service_categories

Marketplace categories (Plumbing, Electrical, Cleaning, Tutoring, Repairs, Tech Support, and any admin adds).

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `name` | varchar, unique | |
| `description` | text, nullable | |
| `icon` | varchar | Emoji shown in category pills |

### services

A provider's individual listing.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `provider_id` | integer, FK → users.id | |
| `category_id` | integer, FK → service_categories.id | |
| `title / description` | varchar / text | |
| `price` | numeric | LKR |
| `city` | varchar | |
| `duration` | varchar | e.g. "1-2 hours" |
| `status` | enum | active / paused / draft |

### bookings

The core transaction: a customer hiring a provider for a service.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `customer_id / provider_id` | integer, FK → users.id | Both reference the same table |
| `service_id` | integer, FK → services.id | |
| `booking_date` | timestamptz | Must be in the future on creation |
| `status` | enum | pending / accepted / rejected / in_progress / completed / cancelled |
| `notes` | text, nullable | |

### reviews

One review per completed booking.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `booking_id` | integer, FK → bookings.id, unique | Enforces one review per booking |
| `customer_id / provider_id / service_id` | integer, FK | Derived from the booking server-side |
| `rating` | integer | 1–5 |
| `comment` | text, nullable | |

### payments

Simulated payment record for a completed booking.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `booking_id` | integer, FK → bookings.id | |
| `customer_id` | integer, FK → users.id | |
| `amount` | double precision | |
| `status` | varchar | pending / completed / failed / refunded |
| `payment_method` | varchar | card / bank / cash |
| `transaction_id` | varchar | Generated on success |

### messages

Direct messages, scoped to one booking's two participants.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `booking_id` | integer, FK → bookings.id | |
| `sender_id` | integer, FK → users.id | |
| `body` | text | |
| `is_read` | boolean | |

### notifications

In-app notifications for booking, review, and message events.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `user_id` | integer, FK → users.id | Recipient |
| `title / message` | varchar / text | |
| `is_read` | boolean | |
| `booking_id` | integer, FK → bookings.id, nullable | |

### provider_availability

A provider's recurring weekly open hours.

| COLUMN | TYPE | NOTES |
| --- | --- | --- |
| `id` | integer, PK | |
| `provider_id` | integer, FK → users.id | |
| `day_of_week` | integer | 0 = Sunday .. 6 = Saturday |
| `start_time / end_time` | time | |

### favorites

A customer's saved services.

| COLUMN | TYPE | NOTES |
| --- | --- | --- |
| `id` | integer, PK | |
| `user_id` | integer, FK → users.id | |
| `service_id` | integer, FK → services.id | |

### service_areas / service_images / service_embeddings

Supporting tables: coverage districts per provider, listing photos, and vector embeddings for AI search.

| COLUMN | TYPE | NOTES |
| --- | --- | --- |
| `service_areas` | district, city, radius_km | FK → users.id (provider), services.id (optional) |
| `service_images` | image_url | FK → services.id |
| `service_embeddings` | embedding (text) | FK → services.id |

### ai_conversations / ai_messages

Chat history for the AI assistant, one conversation per thread.

| COLUMN | TYPE | NOTES |
| --- | --- | --- |
| `ai_conversations` | user_id, title, timestamps | FK → users.id |
| `ai_messages` | conversation_id, role, message | FK → ai_conversations.id |

### review_summaries

Cached AI-generated summary of a service's reviews.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `id` | integer, PK | |
| `service_id` | integer, FK → services.id | |
| `summary` | text | Regenerated on request |

## 3. Normalization

The schema is in Third Normal Form (3NF): every non-key column depends only on its table's primary key, repeating groups are split into their own tables (e.g. `service_images`, `service_areas` instead of array columns), and derived values that must stay in sync (a provider's `avg_rating` / `total_reviews`) are recalculated on write rather than trusted as user input.

## 4. Key Constraints & Integrity Rules

- A `reviews` row is unique per `booking_id` — the database itself prevents a second review on the same booking.
- All foreign keys use `ON DELETE CASCADE` from `users`, so removing an account cleans up its bookings, services, and messages consistently.
- Enum columns (`role`, `status`, booking `status`, verification `status`) are backed by real Postgres enum types, not free-text strings.